Virtual Vending Machine System

מערכת לניהול מכונות מכירה: יצירת מכונות, הוספת מדפים, טעינת מלאי ורכישת מוצרים. הדגש הוא על לוגיקה עסקית ברורה, טיפול עקבי בשגיאות ו-API פשוט לשימוש.

ארכיטקטורה:

המערכת מחולקת לשלוש שכבות:

Domain

מכיל את כל הלוגיקה העסקית: ניהול מדפים ומלאי, בדיקות ולידציה, חריגות דומיין במקום החזרת null

API

קונטרולרים פשוטים שמפעילים את הדומיין בלבד.

נתיבים RESTful ליצירת מכונה, הוספת מדפים, טעינת מלאי, רכישה ושליפה.

כל פעולה מחזירה את מצב המכונה המלא.

Error Handling

טיפול מרכזי בשגיאות באמצעות GlobalExceptionHandler.

מיפוי חריגות לקודי תשובה ברורים (400, 404, 409).

פורמט שגיאות אחיד.

Repository

In-Memory בהתאם לדרישות.

זורק חריגות במקרה של משאב שלא נמצא.

Unit Tests

בודקות את הלוגיקה העסקית:

יצירת מכונה, הוספת מדפים, טעינת מלאי, טעינה מצטברת, רכישה, ומקרי כישלון כמו סוג מוצר לא תואם, קיבולת חריגה, מדף לא קיים ומדף ריק.

Assumptions & Questions

הנחות שבוצעו:

- אין מערכת הרשאות או משתמשים
- אין סליקה אמיתית
- אין בסיס נתונים
- אין צורך בטיפול מתקדם במקביליות
- סוגי המוצרים מנוהלים כ-Enum פשוט

שאלות והחלטות שהייתי מעלה בסביבה אמיתית:

טיפול בשגיאות

בחרתי לא להשתמש ב-try/catch בקונטרולרים, במקום זה יש טיפול גלובלי בשגיאות שמרכז את כל ההמרה של Exceptions לתשובות HTTP במקום אחד. זה הופך את הקונטרולרים לנקיים יותר, ומקל מאוד על תחזוקה והרחבה עתידית.

סוגי מוצרים

סוגי המוצרים מנוהלים כ-Enum כדי לשמור על אחידות ו-Type Safety. אם בעתיד נדרש קטלוג מוצרים מלא (מחירים, שמות מדויקים, תאריכי תפוגה), נוכל להחליף את ה-Enum למבנה נתונים רחב יותר.

בסיס נתונים והיקפים גדולים

המערכת משתמשת ב-In-Memory Repository בהתאם לדרישות. בסביבה אמיתית עם הרבה מכונות נדרש DB אמיתי, אינדקסים, Pagination ו-Rate Limiting. ה-Repository Pattern מאפשר מעבר כזה בלי לשנות את הדומיין.

תחרות וגישה מקבילית

במערכת משחק מרובת משתמשים ייתכן מצב שבו כמה שחקנים מנסים לרכוש את אותו מוצר בו-זמנית. במקרה כזה נדרש מנגנון Concurrency Control (כמו Optimistic Locking) כדי למנוע Race Conditions.

Future Extensions

אפשרויות הרחבה:

- הוספת סוגי מוצרים נוספים
- תמיכה בסליקה אמיתית
- מעבר למסד נתונים
- שכבת שירותים (Application Layer)
- לוגים ומדדים למעקב

Summary

המערכת עומדת בדרישות המשימה: דומיין נקי, API ברור, טיפול שגיאות עקבי, בדיקות יחידה משמעותיות וקוד קריא שניתן להרחבה.